

Performance Testing

Date	0
Team ID	
Project Name	Personalized Networking Assistant
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Pytest, FastAPI TestClient (httpx), Manual UI Testing
Type of Testing	Unit Testing, Functional Testing, API Testing
Target Module / API	Event Analyzer, Topic Generator, Fact Checker, FastAPI Routes
Test Environment	Local Development Environment (Windows OS, Python 3.12, FastAPI, Streamlit)
Test Date	03 July 2026

Step 2: Test Scenarios

S. No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Test Event Analyzer using sample event descriptions	1	2 – 3	Event themes extracted successfully using DistilBERT
2	Test Topic Generator through Streamlit UI	1	2 – 4	Personalized conversation starters generated successfully
3	Test Fact Checker module using Wikipedia API	1	1 – 3	Relevant fact summaries retrieved successfully
4	Test FastAPI routes using Pytest and FastAPI TestClient	1	2	API endpoints responded with expected status codes and outputs

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 4 seconds	Within expected range	Pass	Observed during manual testing
2	Response Time (Max)	< 8 seconds	Within expected range	Pass	No significant delays observed
3	API Availability	100%	All tested endpoints responded successfully	Pass	Verified using FastAPI TestClient and Swagger UI
4	Error Rate	0%	No errors observed	Pass	During manual and API testing
5	CPU Utilization	N/A	Not Measured	N/A	Resource monitoring not performed
6	Memory Utilization	N/A	Not Measured	N/A	Resource monitoring not performed

Step 4: Observations & Analysis

Key Findings:

- Event Analyzer successfully extracted relevant themes from networking event descriptions.
- GPT-2 generated meaningful and context-aware conversation starters.
- Wikipedia API returned concise summaries for fact verification.
- FastAPI endpoints functioned correctly during API testing.
- Streamlit frontend provided a smooth end-to-end user experience.
- All implemented pytest test cases executed successfully.

Bottlenecks Identified:

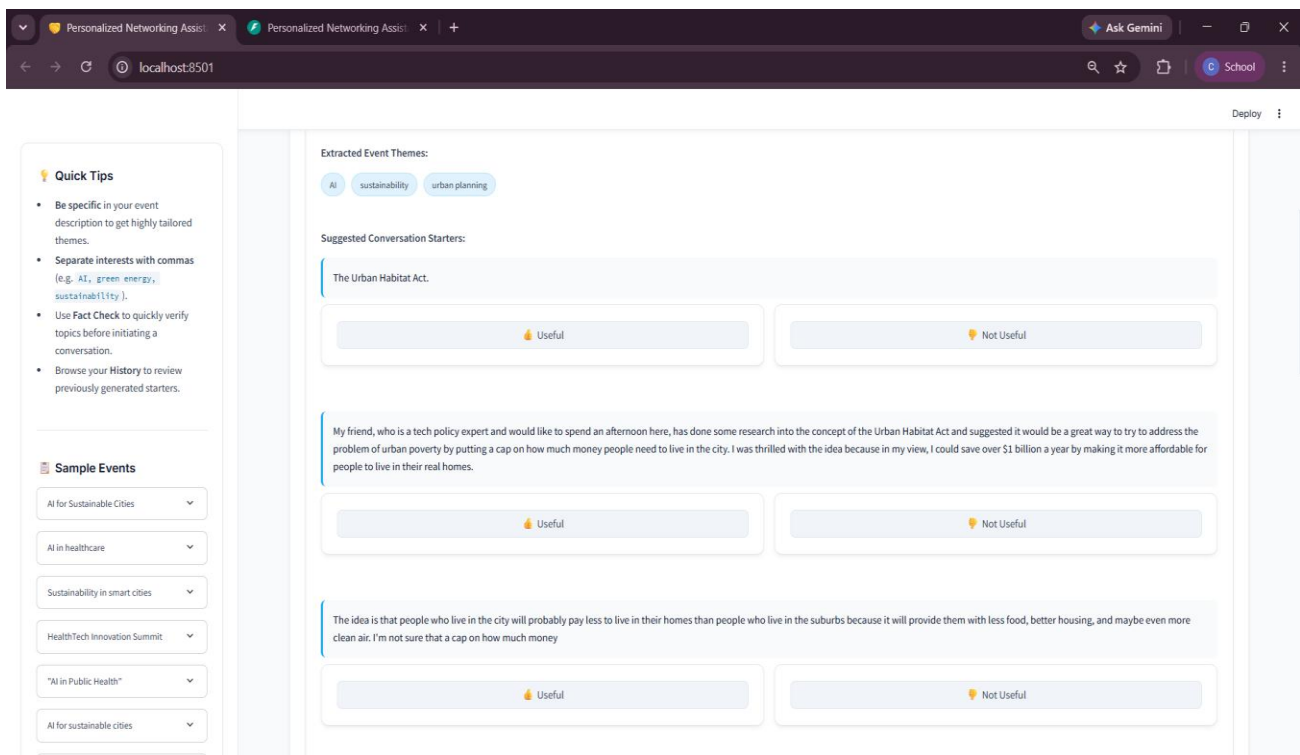
- Initial application startup takes longer because DistilBERT and GPT-2 models are downloaded and loaded into memory on the first run.
- Wikipedia fact verification depends on internet connectivity and API response time.
- GPT-2 generation time increases slightly for longer event descriptions.

Optimization Steps Taken:

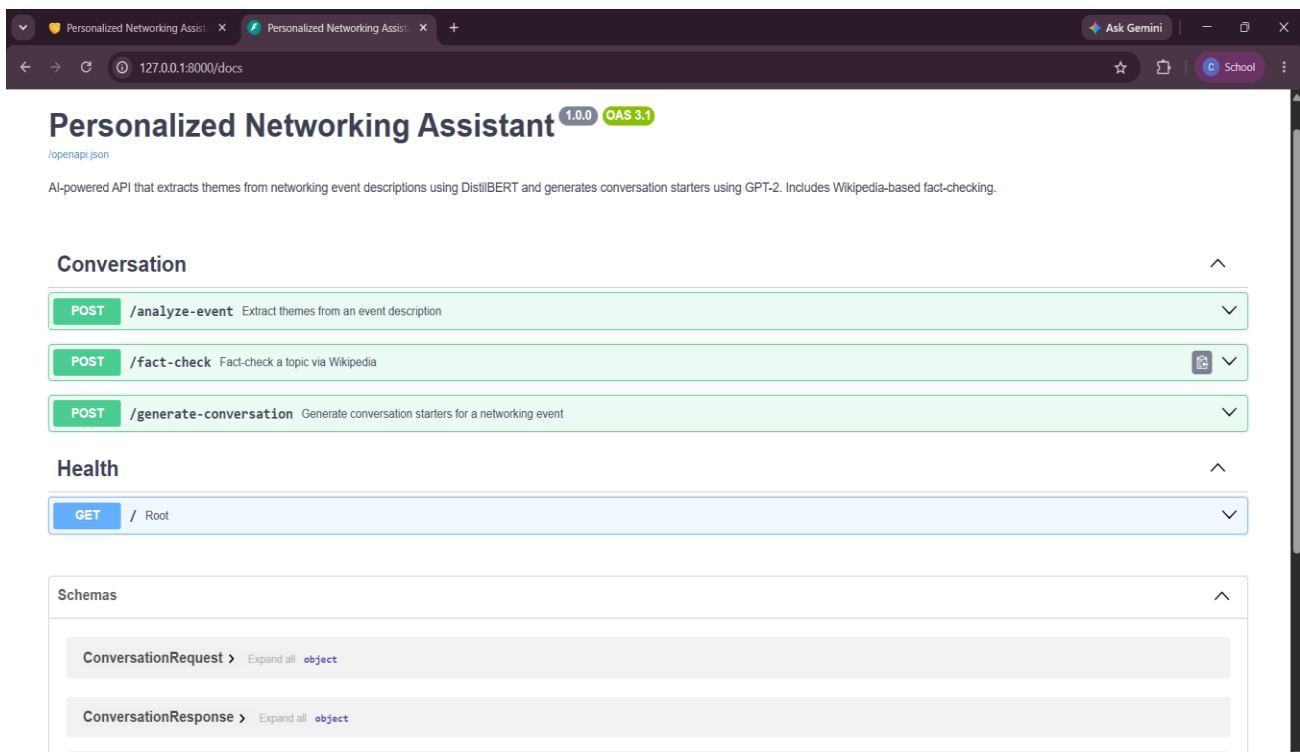
- Used DistilBERT, a lightweight transformer model, for efficient theme extraction.
- Cached downloaded Hugging Face models locally for faster subsequent executions.
- Implemented modular architecture separating frontend, backend, and AI services.
- Added input validation to reduce invalid API requests.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):



Streamlit application displaying generated conversation starters



FastAPI Swagger UI showing successful API execution

```

• (venv) PS C:\SEM5\Internship_Smartbridge\Project> pytest tests/ -v
===== test session starts =====
platform win32 -- Python 3.11.7, pytest-8.3.5, pluggy-1.6.0 -- C:\SEM5\Internship_Smartbridge\Project\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\SEM5\Internship_Smartbridge\Project
plugins: anyio-4.14.1
collected 27 items

tests/test_event_analyzer.py::test_event_analysis_returns_list PASSED [ 3%]
tests/test_event_analyzer.py::test_event_analysis_returns_at_most_three PASSED [ 7%]
tests/test_event_analyzer.py::test_event_analysis_returns_non_empty PASSED [ 11%]
tests/test_event_analyzer.py::test_event_analysis_labels_from_candidates PASSED [ 14%]
tests/test_event_analyzer.py::test_event_analysis_custom_labels PASSED [ 18%]
tests/test_event_analyzer.py::test_event_analysis_short_description PASSED [ 22%]
tests/test_fact_checker.py::test_fact_checker_returns_summary PASSED [ 25%]
tests/test_fact_checker.py::test_fact_checker_returns_correct_extract PASSED [ 29%]
tests/test_fact_checker.py::test_fact_checker_missing_extract PASSED [ 33%]
tests/test_fact_checker.py::test_fact_checker_handles_connection_error PASSED [ 37%]
tests/test_fact_checker.py::test_fact_checker_handles_timeout PASSED [ 40%]
tests/test_fact_checker.py::test_fact_checker_handles_generic_exception PASSED [ 44%]
tests/test_routes.py::test_root_health_check PASSED [ 48%]
tests/test_routes.py::test_generate_conversation_api PASSED [ 51%]
tests/test_routes.py::test_generate_conversation_returns_non_empty_suggestions PASSED [ 55%]
tests/test_routes.py::test_generate_conversation_invalid_request_422 PASSED [ 59%]
tests/test_routes.py::test_generate_conversation_missing_interests_422 PASSED [ 62%]
tests/test_routes.py::test_analyze_event_api PASSED [ 66%]
tests/test_routes.py::test_analyze_event_invalid_422 PASSED [ 70%]
tests/test_routes.py::test_fact_check_api PASSED [ 74%]
tests/test_routes.py::test_fact_check_invalid_422 PASSED [ 77%]
tests/test_topic_generator.py::test_topic_generation_returns_list PASSED [ 81%]
tests/test_topic_generator.py::test_topic_generation_returns_non_empty PASSED [ 85%]
tests/test_topic_generator.py::test_topic_generation_returns_strings PASSED [ 88%]
tests/test_topic_generator.py::test_topic_generation_non_empty_strings PASSED [ 92%]
tests/test_topic_generator.py::test_topic_generation_at_most_three PASSED [ 96%]
tests/test_topic_generator.py::test_topic_generation_empty_interests PASSED [100%]

===== 27 passed in 196.87s (0:03:16) =====

```

Terminal output showing successful execution of pytest